Resolve, Don't Remember: Boosted Semantic Re-Resolution with Calibrated Abstention for In-App Onboarding Guidance

Swastik Satyam
Reg. No. 25BCB0039
Dept. of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
swastik.satyam2025@vitstudent.ac.in

Keshav Yadav
Reg. No. 25BCE2981
Dept. of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
keshav.yadav2025@vitstudent.ac.in

Anas Razy
Reg. No. 25BCB0095
Dept. of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
anas.razy2025@vitstudent.ac.in

Sai Sweta Bal
Reg. No. 25BCB0071
Dept. of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
sai.swetabal2025@vitstudent.ac.in

Viveka T
Reg. No. 25BCB0152
Dept. of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India
viveka.t2025@vitstudent.ac.in

Jagadeeswari M
Faculty Guide
Dept. of Mathematics
Vellore Institute of Technology, Vellore, India
jagadeeswari.m@vit.ac.in

***Abstract*—In-app guidance products, known as digital adoption platforms (DAPs), anchor each walkthrough step to a page element using a selector captured at authoring time. After a redeploy such an anchor either fails silently or, worse, points confidently at the wrong control. We study this problem with a controlled benchmark of 30 real and fixture page states under 8 seeded perturbation operators, comparing our onboarding copilot against six locator strategies documented by commercial DAPs and the web-testing literature, including a Similo-style weighted-similarity matcher. Our first resolver (v1, hand-weighted semantic scoring) is correct in 72.2% of composite-redeploy resolutions, ahead of a role+name locator (66.1%) but well behind the Similo-style matcher (93.6%); we report this plainly. A learned listwise ranker over 20 locator and semantic features, with context features that compare each candidate with its competitors (v2), reaches 94.4% correct and 5.6% wrong on held-out pages (95% CI 92.3–96.5), statistically tied with the Similo-style matcher ($+0.8$ points, CI -1.3 to 3.0), and stays near 98% correct under label edits on a perturbation type withheld from training. A gradient-boosted ranker on the same features (v3) reaches 96.3% correct and 3.7% wrong on held-out pages (CI 93.9–98.2), significantly ahead of the Similo-style matcher ($+2.7$ points correct, CI 1.1 to 4.6; -2.7 points wrong). Margin-gated abstention cuts the wrong-highlight rate of our weaker v1 resolver by 40%, with smaller gains for strong matchers. On the real before/after data released with Similo (40 sites, 598 targets) a boosted ranker ties the original Similo code, and after a failure analysis, wording-robust features plus joint assignment give a modest but statistically significant gain (94.0% against 90.1% on 37 sites with identical targets), while a margin gate cuts its wrong-highlight rate to 2.6% against 9.9%. We also report the language-model compile-once architecture, measured runtime costs, and a defect the evaluation exposed in our own code.**

***Index Terms*—digital adoption platform, onboarding, web element localization, selective prediction, abstention, large language models, human-computer interaction.**

I. Introduction

Software products lose users who never reach their first valuable action. Digital adoption platforms try to prevent this by drawing tooltips and spotlights over an existing application without changing its source. The core operation is simple to state and hard to do well: given a step such as "click *Invite* in the Members section", find that element on the live page. The vendors we reviewed (WalkMe, Whatfix, Pendo, Appcues, Userpilot, Userflow) describe the same basic answer in their help pages: capture an element or a selector while the flow is built and look it up again at run time [2]–[6].

Locators are known to be fragile. Hammoudi et al. catalogued 1,065 breakages of record/replay web tests across 453 versions of web applications and found locator problems among the main causes [7]. Guidance has an additional hazard that tests do not. A broken test fails loudly, but a mis-anchored spotlight points confidently at the wrong control, and the user trusts it.

We study a copilot built around three ideas. First, a step carries a *semantic descriptor* (role, accessible name, section, ordinal, and so on) that is re-scored against every live element on every tick, so nothing is cached that a re-render can invalidate. Second, a language model is used *once per goal, offline*, as a compiler that emits a validated plan; at run time everything is deterministic and local. Third, the system is allowed to *abstain*: when the evidence for a single element is weak, it shows text instead of a spotlight.

Contributions.

- **A controlled robustness benchmark for guidance anchors.** Eight seeded perturbation operators on 30 real and fixture page states, with ground truth carried through every perturbation and metrics that separate *wrong* highlights from *abstentions* (Sec. IV).
- **An honest head-to-head.** Our hand-weighted v1 resolver beats documented DAP locator strategies but *loses* to a Similo-style weighted-similarity matcher; we show where and why.
- **Learned local rankers (v2, v3).** A listwise linear ranker (v2) ties the strongest baseline on held-out pages and generalises to an unseen perturbation type; a gradient-

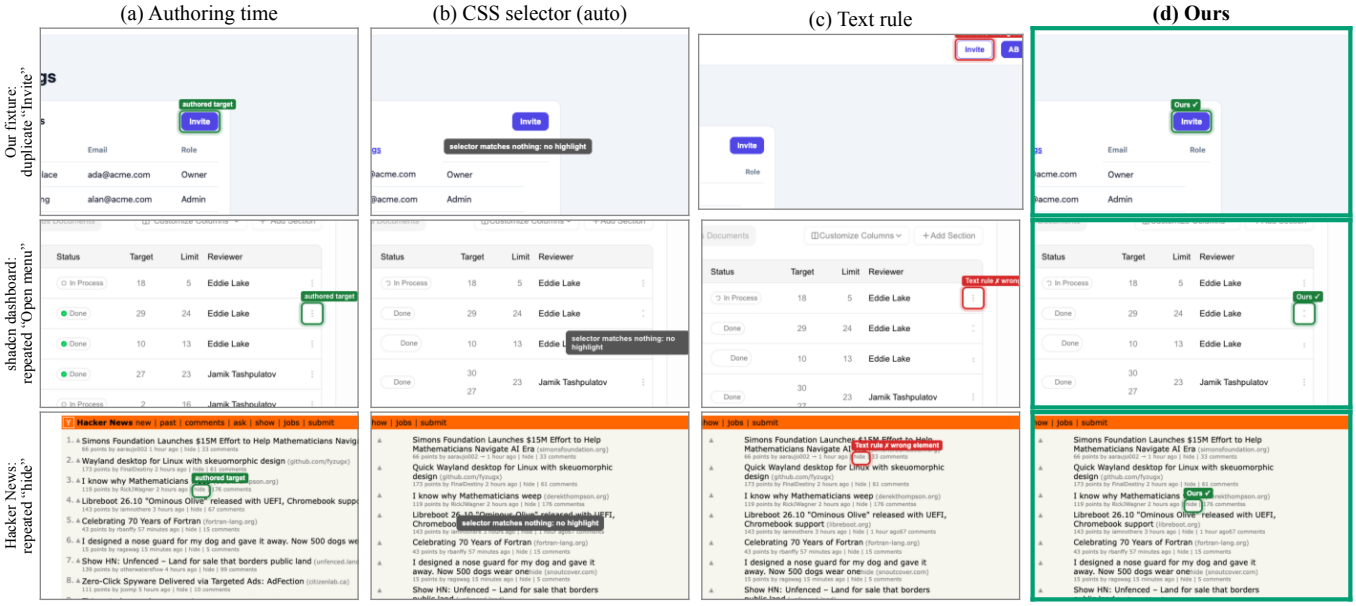


Fig. 1. **Same step, same redeployed page, four anchoring strategies.** Each row authors a step on the pristine page (a), then re-resolves it after a composite redeploy (class/ID renaming, wrapper drift, reordering, copy edits, injected duplicates; ρ=0.5). (b) A CSS selector generated by an off-the-shelf tool [1] matches nothing, so the step silently fails. (c) A text rule finds the first control with the same label, which is a *different* control (red). (d) Our v1 resolver re-scores every live element against a semantic descriptor and picks the intended one (green). Examples are *selected* to show the three behaviours side by side; aggregate results over 60,475 resolutions are in Figs. 3–4 and Table V.

boosted ranker (v3) on the same candidate-context features is significantly better than every baseline we ran, including a Similo-style matcher.

- **Calibrated abstention and limits.** A margin gate calibrated on held-out pages, its measured benefit for each matcher, and an account of limits including a polling defect found in our own SDK.

II. Related Work

Commercial DAPs. Table I summarises the targeting behaviour that vendors document. WalkMe describes capturing an element by ID, position, text, or jQuery rule [2]; its help pages list re-selecting the element or relaxing matching rules as the fix when an element is not found [8], [9]. Whatfix labels captured elements strong or weak and suggests adding selector or text rules [10]. Pendo offers suggested, parent, or custom CSS rules [3], and its community reports guides whose target matches several elements [11]. Appcues stores a CSS selector at build time and asks authors to use stable attributes or edit the selector [4], [12]. Userpilot documents errors for dynamic IDs and iframes [5], [13], and Userflow asks for a manual selector when an element is not found [6]. In every case the documented remedy is an *author edit*. Whatfix has also announced LLM-assisted authoring and agents that adapt to interface changes [14], [15], which makes it the closest documented competitor. Documentation shows what a vendor states, not what it ships; we therefore do not benchmark vendor products, only the strategies they document (Sec. IV).

Robust element localization in web testing. The testing community has studied locator robustness for over a decade. ROBULA+ generates XPath locators that reduce fragility relative to absolute paths [16]. Similo goes further and scores candidates by a weighted similarity over several locator attributes, locating the target in 89% of 801 cases against 73% for a baseline in the authors' study [17]. Similo is the closest relative of our scorer, so we include a Similo-style arm in our benchmark (our simplified reimplementation of weighted attribute similarity, not the authors' code), and we make no claim to have invented weighted multi-attribute matching. Later work extends Similo with visual-overlap groups (VON Similo) [18], an LLM re-ranking step [19], and a hybrid of the two evaluated on a 23 times larger benchmark [20]; a 2026 dataset collects reproducible locator breaks from open-source test suites [21]. Their larger benchmarks and the LLM and hybrid variants are not released in a form we could run (the replication repository for [20] omits the scraped site data), so we compare against the original Similo only. Our setting differs in three ways: the cost of a wrong answer is a confident misdirection rather than a failed test, so abstention is a first-class output; descriptors carry *section context* derived from landmarks; and the resolver is part of a system that compiles, validates, caches, and repairs plans.

LLM web agents. Mind2Web and SeeAct ask a language model to choose a page element for each action [22], [23]. SeeAct reports that grounding the model's plan to the right element is the main bottleneck. Putting the model on the hot path also adds a model call, with its latency and cost, to every step, and exposes every step to hallucination [24]. We take the opposite stance: the model runs once per goal, offline, picks among deterministically extracted candidates by id, and

TABLE I
TARGETING BEHAVIOUR DOCUMENTED BY SIX DAPS, AND OURS. "REMEDY" IS THE FIX THE DOCUMENTATION PRESCRIBES.

System	Targeting	Failure and documented remedy	AI features documented
WalkMe [2], [8]	Captured element: ID, position, text, jQuery	Element absent: re-select or relax matching	not reviewed
Whatfix [10], [14]	Captured element, strong/weak, CSS/jQuery	Weak element: add selector or text rule	LLM authoring, adaptive agents
Pendo [3], [11]	Suggested, parent, or custom CSS	Multiple matches: parent or custom CSS	not reviewed
Appcues [4]	CSS selector saved at build time	Not unique/found: stable attributes or edit	not reviewed
Userpilot [5], [13]	Auto CSS selector, optional text	Dynamic IDs, iframes: manual selector	not reviewed
Userflow [6]	Captured CSS class, ID, attribute	Not found: manual selector	not reviewed
Ours	Semantic descriptor re-scored every tick; confidence gate	Below threshold: text-only guidance; telemetry; automatic recompile	One validated LLM call per goal, offline

is validated.

Reliability and abstention. The reject option [25] formalises the error–reject trade-off that our gates navigate. We adopt its evaluation view: report the correct, wrong, and abstained shares, and compare methods on the correct–wrong plane rather than on a single accuracy number.

Position. To our knowledge, none of the DAPs whose documentation we reviewed documents the combination of semantic re-resolution on every tick, confidence-gated abstention, and a self-repairing compiled plan, and we are not aware of a published controlled evaluation of guidance-anchor robustness. Each element of the combination has precedent; the contribution is the combination and its measured behaviour.

III. METHOD

A. Overview

Fig. 2 shows three planes. The *runtime plane* runs inside the host page: it extracts a semantic view of the page, re-resolves the current step's target, and draws a spotlight or text-only card. The *intelligence plane* turns a free-form goal and the tenant's documents into a validated plan with a single structured model call, cached per goal. The *control plane* records events and evicts plans that keep failing. The compiled plan ships whole at session start, so the model is never called while a user is mid-guide.

B. Semantic extraction

Browsers expose no accessibility-tree API to in-page scripts, so the SDK computes one. For each interactive element it derives an ARIA role (explicit or implicit from the tag), an accessible name using a subset of the W3C algorithm [26], a *section label* (the accessible name of the nearest enclosing landmark), an *ordinal* (index among elements sharing role and name), visibility, bounds, and a stable id (a hash of role, name, test id, section, structural path, and ordinal). Extraction keeps at most 250 elements per snapshot, visible-first; on pages with more than 500 interactive elements it first pre-selects the viewport-nearest candidates with one batched layout read.

C. Descriptor scoring

A step carries a descriptor d holding the signals recorded when the element was authored. Let $I(d)$ be the signals that d specifies, $f_i(e, d) \in [0, 1]$ the match strength of signal i on candidate e, and w_i its weight. The score is normalised by the weight of the specified signals only, so a sparse descriptor is not penalised for omitting signals:

$$S(e, d) = v(e)\,\frac{\sum_{i \in I(d)} w_i\, f_i(e, d)}{\sum_{i \in I(d)} w_i}, \tag{1}$$

where $v(e)$ is 0.5 for a hidden element and 0.7 for a disabled one (factors multiply). The weights in the shipped code are test id 1.0, name 0.9, `aria-label` 0.8, internal id 0.7, text 0.6, role 0.5, section 0.5, ordinal 0.3, ancestry 0.3, and bounding-box proximity 0.15. The best candidate is $e^* = \arg\max_e S(e, d)$.

The shipped code has two entry points. The *plan executor* matches *authored criteria* (role, exact name, optional section hint and test id) and draws a spotlight only if $S_{(1)} \geq \tau = 0.6$ *and* the section hint matches (a hard constraint):

$$\text{highlight} \iff S_{(1)} \geq \tau \ \wedge\ \text{section}(e^*)\ \text{matches}. \tag{2}$$

The *runtime matcher* re-resolves the full descriptor of a displayed target every frame with a lower floor of 0.4. We benchmark both paths. The existing ambiguity flag (top-two gap below 0.08) only requests an optional AI tie-break; it does not suppress the highlight.

D. Calibrated abstention (proposed)

A confidence floor alone cannot tell a clear winner from a near-tie between two plausible elements. We therefore evaluate a *margin gate*, which is *not* in the shipped code:

$$\text{highlight} \iff S_{(1)} \geq \tau \ \wedge\ S_{(1)} - S_{(2)} \geq \delta. \tag{3}$$

We select δ by two-fold cross-fitting over pages: for each fold, δ is the smallest value in $\{0.02, 0.05, 0.08, 0.1, \dots\}$ that keeps the wrong-highlight rate at or below 5% on the *other* fold's

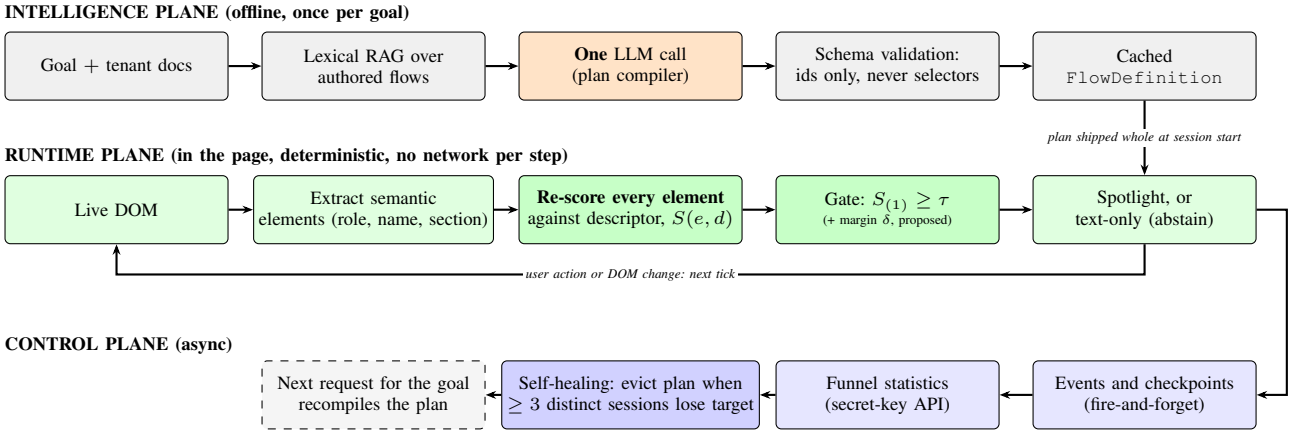


Fig. 2. System overview. The language model appears exactly once, offline. Everything on the per-step path is a local, deterministic computation; telemetry flows back asynchronously and triggers recompilation when a cached plan keeps losing its target.

composite-redeploy trials, and the gate is applied to the held-out fold. Both folds selected δ=0.05. This is the reject option of [25] applied to element resolution.

E. Learned ranker (v2)

The v1 weights were set by hand. Borrowing from learning-to-rank, v2 treats resolution as a listwise choice among candidates. For each target we take all elements with the authored tag, keep the 15 best by a cheap prefilter, and describe every candidate with 20 features in $[0, 1]$: Similo-style locator attributes (class, id, label, href, text, path, position, size, parent class, neighbour text) and our semantic signals (role, exact and fuzzy name, section, ordinal, visibility, test id). *Context features* append, for each feature, the candidate's value minus the maximum and minus the mean over the candidate list, a one-layer analogue of the cross-candidate comparison that attention provides. Scores are linear, $s_j = \mathbf{w}^\top \phi_j$, and trained with a softmax cross-entropy over the list by L-BFGS. Inference is 60 multiply-adds per candidate, so once candidates are extracted the ranking costs microseconds; extraction, not ranking, dominates latency (Sec. V). We deliberately did *not* use a transformer: with 20 features and 15 candidates a linear listwise model is enough, exportable as one weight vector, and free of inference-time dependencies. Semantic text embeddings are the natural next step for wording changes.

A boosted variant (v3). A non-linear model can capture interactions that a linear score cannot, for example that a text match is decisive only when the position also agrees. v3 fits a gradient-boosted tree classifier (250 trees, 31 leaves, `HistGradientBoosting`) on the same context-augmented features, scoring each candidate pointwise and taking the argmax. In-browser latency of tree inference was not measured here; the model is small, but v3 is not yet integrated into the SDK.

Evaluation protocol for v2 and v3. To avoid fitting the benchmark, we use four-fold cross-validation over *pages*: train on two folds, calibrate the margin gate δ on a third, and test on the fourth. Separately, a leave-one-operator-out run withholds each perturbation type from training and tests on it.

F. Compile-once planning and self-healing

Given a goal, the server tries authored flows (lexical retrieval), then a cached compiled flow, then makes *one* structured model call (Gemini 2.5 Flash through the Vercel AI SDK, cf. [27]) grounded in knowledge chunks. The model sees a deterministic shortlist of at most eight candidates and returns an *id*; every output is validated against a Zod schema, and an unknown id is rejected. A made-up identifier therefore cannot be rendered, but a wrong *valid* candidate can; the scoring gates act as an independent deterministic check. A cached plan is evicted when k=3 distinct sessions report a lost target. If n sessions fail independently with probability p, then $P_{\text{evict}} = \Pr[X \geq k]$ for $X \sim \text{Bin}(n, p)$; for a broken plan (p=0.5, n=10) this is 0.945, and for a healthy one (p=0.01, n=100) it is 0.079 (analytic; the eviction logic is covered by unit tests, but we did not measure eviction on live traffic).

G. Snapshot codec

Runs of at least three structurally identical siblings are sent as one template plus per-instance deltas. If a group of m elements has template size s and per-instance delta d_Δ, the compression ratio is $1/m + d_\Delta/s$. We measure real ratios in Sec. V.

IV. Experimental Protocol

A. Corpus and ground truth

We use 30 page states (Table II): three states of our own hostile fixture (duplicate buttons, portal modal) and 27 public pages spanning admin dashboards, documentation, forms, and content sites (a 28th snapshot, the Internet Archive home page, yielded no eligible targets and is excluded). Each page is captured once as a self-contained MHTML snapshot with scripts removed, so every run starts from identical bytes, and is loaded in desktop Chrome. Before any perturbation, every element receives a hidden `data-gt` index. Perturbations carry

TABLE II
CORPUS. "TARGETS" IS THE NUMBER OF ELIGIBLE TARGET ELEMENTS (VISIBLE, ENABLED, NAMED; *of which* UP TO 10 ARE SAMPLED); "DUP." COUNTS THOSE SHARING ROLE AND ACCESSIBLE NAME WITH ANOTHER ELEMENT.

Page state	Type	Targets	Dup.
Our fixture: dashboard	Fixture	14	0
Our fixture: members settings	Fixture	14	2
Our fixture: invite modal open	Fixture	17	2
AdminLTE 3 dashboard demo [28]	Admin UI	33	6
AdminLTE general forms [29]	Forms	73	23
AdminLTE data tables [30]	Admin UI	44	10
Angular home [31]	Docs	48	2
Bootstrap checkout example [32]	Forms	25	0
Bootstrap dashboard example [33]	Admin UI	18	0
Bootstrap forms overview [34]	Forms	167	39
Bootstrap navbar component docs [35]	Docs	247	110
Bootstrap pricing example [36]	Web app	23	0
Bootstrap sign-in example [37]	Forms	5	0
Django documentation [38]	Docs	226	38
Flask documentation [39]	Docs	249	45
GitHub repository page [40]	Web app	201	54
Hacker News front page [41]	Content	187	57
MDN `<button>` reference [42]	Docs	250	38
Next.js documentation [43]	Docs	108	39
PyPI home [44]	Web app	54	7
Python 3 documentation [45]	Docs	78	47
React documentation home [46]	Docs	122	60
shadcn/ui dashboard example [47]	Admin UI	118	55
shadcn/ui tasks example [48]	Admin UI	89	55
Svelte home [49]	Docs	126	9
Tailwind CSS installation docs [50]	Docs	234	35
TodoMVC (React) [51]	Web app	12	2
Vue.js home [52]	Docs	56	42
W3C home [53]	Content	47	7
Wikipedia main page [54]	Content	229	24

the attribute with the node; clones injected as duplicates do not. *No method reads it*: it is used only to score outcomes. A resolution is *correct* if it returns the element with the authored index, *wrong* if it returns another element, and an *abstention* if it returns nothing.

B. Authoring and perturbations

For each page we author 10 targets once, on the pristine DOM, taking the descriptor that the SDK derives from the element (as a recorder would) and, for the criteria path, a role, exact name, and the element's own section and test id when present. Up to three targets per page are chosen among elements that have a same-named twin, so the hard cases are always represented. Table III lists the operators; each takes an intensity $\rho \in \{0.1, 0.25, 0.5, 0.75, 1\}$ that sets the fraction of eligible nodes affected, and each is run with 5 seeds. Class and ID renames also rewrite the page's stylesheets so the layout is preserved. The composite *redeploy* mimics a release that changes several things at once.

C. Methods

Table IV lists the arms. The first seven are *our implementations of the strategies that the DAP documentation and testing literature describe*; they are not the vendors' products, whose internals we cannot inspect. The CSS arm uses an independent open-source selector generator [1] rather than code we wrote.

TABLE III
PERTURBATION OPERATORS.

Operator	Action	Probes
Class rename	Rename CSS classes (and stylesheet rules)	class selectors
ID randomisation	Replace ids; fix `for`, `aria-*`, `href#`	id selectors
DOM reorder	Shuffle siblings of random parents	positions, ordinals
Wrapper drift	Insert layout-neutral wrappers and hidden siblings	paths, nth-child
Copy drift	Suffix, prefix, upper-case, or pluralise labels	text, name rules
Dup. (other sect.)	Clone the target into another container	non-unique matches
Dup. (adjacent)	Clone the target next to itself	inherent ambiguity
Composite redeploy	Classes ρ, ids ρ, wrappers 0.5ρ, copy 0.35ρ, reorder 0.3ρ, dup. 0.3ρ	all of the above

TABLE IV
METHODS. BASELINES EMULATE DOCUMENTED STRATEGIES; THEY ARE NOT VENDOR PRODUCTS.

Arm	Resolution rule	Family
CSS selector	generated by `finder` [1], DOM lookup	Appcues, Userpilot, Userflow, Pendo
XPath	absolute positional path	position capture
Text rule	first same-tag element with identical text	WalkMe, Whatfix text rules
Selector chain	id → CSS → text → XPath, first hit	multi-rule fallback
Role+name	first element with same role and accessible name	Playwright [55]
Role+name (strict)	as above, abstain if not unique	Playwright strict mode
Similo-style	argmax of weighted attribute similarity (tag, class, id, label, href, text, path, position, size); never abstains	Similo [17]
Ours v1 (criteria)	Authored criteria, Eq. (2)	shipped plan executor
Ours v1	Full descriptor, Eq. (1), τ=0.6	shipped matcher
Ours v2	Learned listwise ranker, Sec. III-E, page-held-out	proposed
Ours v3	Boosted-tree ranker, same features, page-held-out	proposed
+ margin	Eq. (3), held-out δ	proposed

D. Metrics and statistics

We report the correct rate C, wrong-highlight rate W, and abstain rate A ($C{+}W{+}A = 1$), plus precision $C/(C{+}W)$. Trials within a page are correlated, so confidence intervals use a *cluster bootstrap over pages* [56], not a binomial interval over resolutions. Paired differences resample the same pages for both methods. Headline numbers pool the composite redeploy at $\rho \in \{0.5, 1\}$ (n=1,400 resolutions per method). The benchmark records 1,209,500 outcomes in total.

V. RESULTS

A. Main comparison

Table V and Fig. 4 compare all arms under composite redeploys. The structural baselines (CSS, XPath) rarely survive a redeploy but *fail loudly*: they abstain in 82.5% and 86.8% of cases and are wrong in about 5–6%. The always-answer baselines that look at text or names (text rule, selector chain,

role+name) are correct in at most 66.1% and wrong in 21.4–34.2% of cases.

Our v1 resolver beats those but not the strongest baseline. It is correct in 72.2% and wrong in 9.7% (CI 7.5–12.3), better than role+name by 6.1 points correct and 11.7 points fewer wrong (both CIs exclude zero). But the Similo-style weighted-similarity matcher, which blends many locator attributes, is correct in 93.6% and wrong in 6.4%, i.e., -21.5 points better than v1 on correct (CI -27.6—16.2). v1 matches names exactly and leans on a positional ordinal hint, and it uses few of the attributes (class, href, path, position, neighbours) that survive a redeploy.

Our learned v2 ranker closes the gap. Trained on other pages only, v2 is correct in 94.4% (CI 92.3–96.5) and wrong in 5.6% (CI 3.5–7.7). Against the Similo-style matcher the paired difference is $+0.8$ points correct (CI -1.3 to 3.0) and -0.8 points wrong: a statistical tie, not a win (v3, below, does better). Against v1 it is a large gain, and against role+name it is $+27.5$ points correct. **A boosted ranker (v3) goes further.** With the same features, v3 is correct in 96.3% (CI 93.9–98.2) and wrong in 3.7% (CI 1.8–6.1). Against the Similo-style matcher the paired differences are $+2.7$ points correct (CI 1.1–4.6) and -2.7 points wrong (CI 1.1–4.6), both excluding zero, so on this benchmark v3 is significantly better than every baseline we ran. The margin gate changes little for v2 (93.8% correct, 5.2% wrong), because a strong ranker leaves few near-ties to abstain on.

B. Where we win, and where we do not

Table VI lists C/W per operator at ρ=1 (v2 and v3 are page-held-out). Three patterns stand out. First, label edits (copy drift) cut v1 and the role+name locator to roughly a third correct, while v2 and the Similo-style matcher stay near 98% because they rely on many non-text signals and v2 has fuzzy name features; structural selectors are unaffected. Second, injected duplicates are the hardest case for every method: with a same-named twin present, role+name is wrong in 57% of composite-redeploy resolutions, v1 in 20%, Similo-style in 16%, and v2 in 12%. Third, DOM reordering is where the structural baselines collapse and the matchers that carry context hold up. In a *leave-one-operator-out* run (each operator withheld from training), v2 stays at 94% correct under reordering, 98% under copy drift and 93% under the composite redeploy, so the learned weights are not tied to the operators they were fit on.

C. Risk–coverage and the margin gate

The shipped confidence floor barely moves v1's operating point: between τ=0.4 and 0.6 its correct rate changes by about one point. A margin gate does more. For v1 it lowers the wrong-highlight rate from 9.7% to 5.9% (CI 3.8–8.1) at a modest cost in coverage. For the strong matchers the gain is small, because there is little error left to remove (Similo-style: 6.4% to 5.0%; v2: 5.6% to 5.2%). Fig. 6 places every method on the correct–wrong plane: abstention is most valuable for weak matchers and is a cheap safety net for strong ones.

D. Ablation

Table VII removes signals from the v1 descriptor. Context matters: role and name alone raise the wrong-highlight rate to about 19%. But removing the *section and ordinal hints together* slightly *improves* v1's pooled result, because the ordinal is positional and brittle under reordering and duplicate injection. This was an early signal that v1's hand-set weights were wrong, and it motivated learning them (Sec. III-E); the learned v2 weights down-weight neither hint to zero but rely mostly on position, href, text, and neighbouring text.

E. Real before/after benchmark: the Similo replication package

Synthetic perturbations are a limitation, so we also ran on the *real* dataset released with Similo [17]: 48 websites with the target element in an old version and all candidates in the new version, with the authors' own per-site outputs for XPath, Selenium IDE, ROBULA+, Montoto, Multilocator, and Similo from their original code. Eight sites ship without their element files, leaving 40 sites and 598 targets; we recompute every published baseline on exactly this subset. Our rankers use the dataset's locator attributes as features, are trained on other sites only (five folds by site), and CIs are bootstrapped over sites.

Table VIII gives the result. **We do not beat the original Similo here.** Our boosted ranker is correct in 88.9% against 90.1% for Similo (-1.2 points, 95% CI -5.1 to 3.1), a tie, and well above ROBULA+, Montoto, Selenium IDE, and Multilocator. The margin gate trades coverage for safety: it cuts wrong highlights from 9.9% to 5.4% (-4.6 points, CI -9.0 to -1.2) at the cost of abstaining on 11.7% of targets and -7.2 points of correct rate. Feeding each confirmed outcome back into the matcher (an online update after every guided step, evaluated predict-then-learn) improved the linear ranker by only 0.2 points (CI 0.0 to 0.6): streams of about 15 targets per site are too short for the feedback loop to matter, so we do not claim a benefit. A further idea that does help is *joint assignment*: because each new element can match at most one old target on a site, solving all of a site's targets together with the Hungarian algorithm raises the linear ranker from 91.6% to 92.7% correct ($+1.2$ points, CI 0.4 to 2.0). Against Similo's 90.1% this is $+2.7$ points with an interval (-0.4 to $+6.8$) that includes zero. **Failure-driven features.** A failure analysis of the remaining errors showed that most (78%, 40 of 51) were targets whose text had changed or vanished, which string-overlap features cannot handle. Adding wording-robust features, namely normalised compression distance over all attributes and token-level overlap of class, href, text, neighbours, and labels, lifts the linear ranker, and with joint assignment it beats Similo on identical targets. To make the comparison exact we restrict to the 37 sites where our parsed target count equals Similo's tabulated count (548 targets; three sites differ because the authors filtered some targets). There the joint ranker is correct in **94.0% against Similo's 90.1%** ($+4.0$ points, 95% CI $+0.4$ to $+8.3$ over sites; one-at-a-time 92.5%). With the margin gate it is wrong on only 2.6% of targets (abstaining on 8.6%) against Similo's 9.9% (-7.5

TABLE V

COMPOSITE REDEPLOY, POOLED OVER $\rho \in \{0.5, 1\}$ (PERCENT; 95% CLUSTER-BOOTSTRAP CIS OVER PAGES IN BRACKETS). C: CORRECT HIGHLIGHT; W: WRONG HIGHLIGHT; A: ABSTAIN; PREC.: $C/(C+W)$. BOLD MARKS THE BEST C AND THE LOWEST W; THE LOWEST W IS ACHIEVED BY ABSTAINING OFTEN. †PROPOSED, NOT SHIPPED; δ CHOSEN ON HELD-OUT PAGES.

Method	$C \uparrow$		$W \downarrow$		A	Prec.
CSS selector (auto)	11.3	[8.7, 14.0]	6.2	[3.7, 9.0]	82.5	64.4
XPath (absolute)	8.5	[6.0, 11.2]	4.6	[2.9, 6.5]	86.8	64.9
Text rule	57.1	[50.9, 62.8]	32.4	[26.0, 40.0]	10.4	63.8
Selector chain	58.2	[53.0, 63.2]	34.2	[28.6, 40.1]	7.6	63.0
Role+name (first)	66.1	[60.9, 71.2]	21.4	[17.0, 26.4]	12.6	75.6
Role+name (strict)	49.8	[44.4, 55.2]	**3.0**	[2.1, 4.2]	47.2	94.3
Similo-style (our reimpl.)	93.6	[90.8, 96.0]	6.4	[4.0, 9.2]	0.0	93.6
Similo-style + margin gate†	92.0	[89.2, 94.8]	5.0	[2.7, 7.7]	3.0	94.9
Ours v1, authored criteria	29.3	[21.6, 38.4]	5.0	[2.5, 7.9]	65.7	85.5
Ours v1 (hand-weighted)	72.2	[67.0, 76.6]	9.7	[7.5, 12.3]	18.1	88.2
Ours v1 + margin gate†	70.7	[65.9, 75.1]	5.9	[3.8, 8.1]	23.4	92.3
Ours v2 (learned ranker)	94.4	[92.3, 96.5]	5.6	[3.5, 7.7]	0.0	94.4
Ours v2 + margin gate†	93.8	[91.5, 96.0]	5.2	[3.3, 7.2]	1.0	94.8
Ours v3 (boosted ranker)	**96.3**	[93.9, 98.2]	3.7	[1.8, 6.1]	0.0	96.3
Ours v3 + margin gate†	96.1	[93.7, 98.0]	3.7	[1.7, 6.0]	0.2	96.3

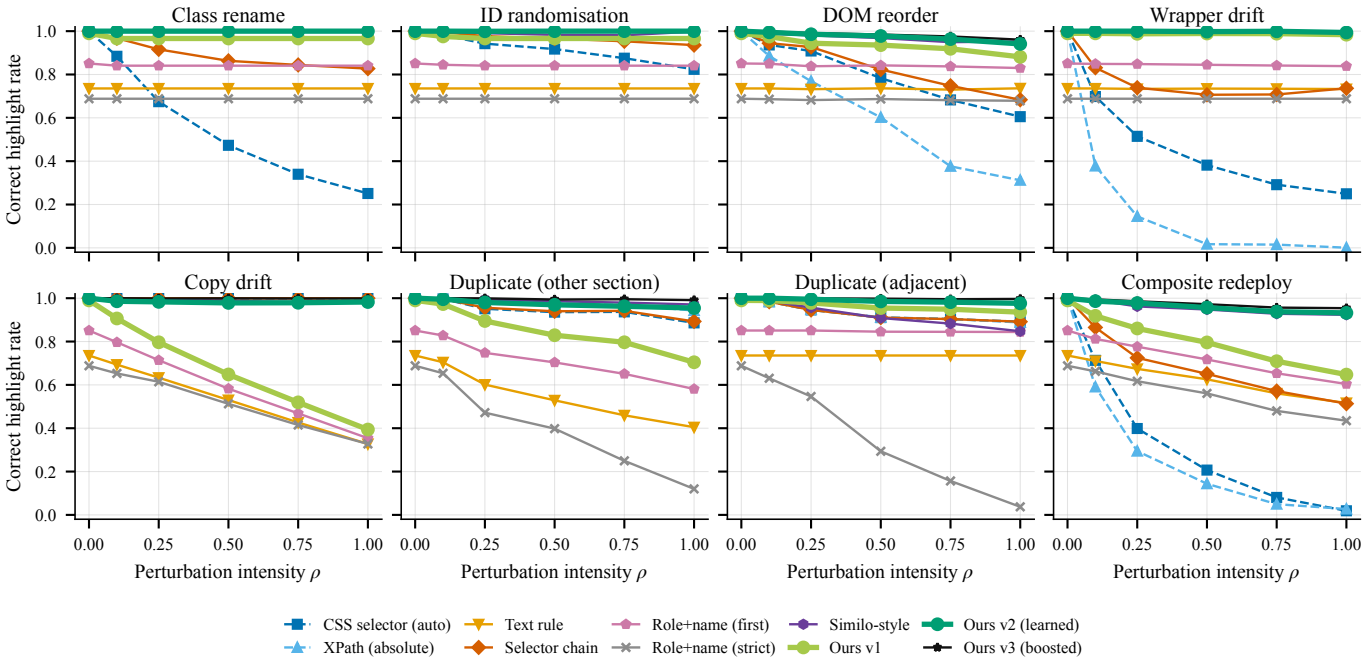


Fig. 3. Correct-highlight rate against perturbation intensity ρ for each operator (identity at ρ=0). Structural baselines (dashed blue) are unaffected by copy drift but collapse under class, wrapper, and composite changes. Our v1 resolver (light green) degrades gracefully but loses ground under copy drift; the learned v2 ranker (dark green) and the Similo-style matcher stay near the top throughout.

points, CI -12.5 to -4.0), at a correct rate (88.9%) statistically indistinguishable from Similo's. Because the features were motivated by error analysis on these same sites, we checked replication on disjoint halves of the sites: the gain is $+3.8$ points on one half (19 sites) and $+4.4$ on the other (18 sites), each with a wider interval because of half the data, and in both halves 80–83% of failures are changed-text targets, so the mechanism replicates. All fits use held-out sites. The remaining limit is that this is a single dataset. We also tried pretraining on 25,000 synthetic-benchmark groups using the nine features the two datasets share: this gave 86.6% correct against 85.6% without pretraining, both below Similo's 90.1% (a negative result we keep, since the shared-feature subset discards the signals that make Similo strong).

What did not help (a broad sweep). To test whether the 94.0% is a modelling ceiling, we tried a wide set of ideas on the 37 matched sites, all with held-out sites. A set-transformer that attends over the candidate set, a listwise

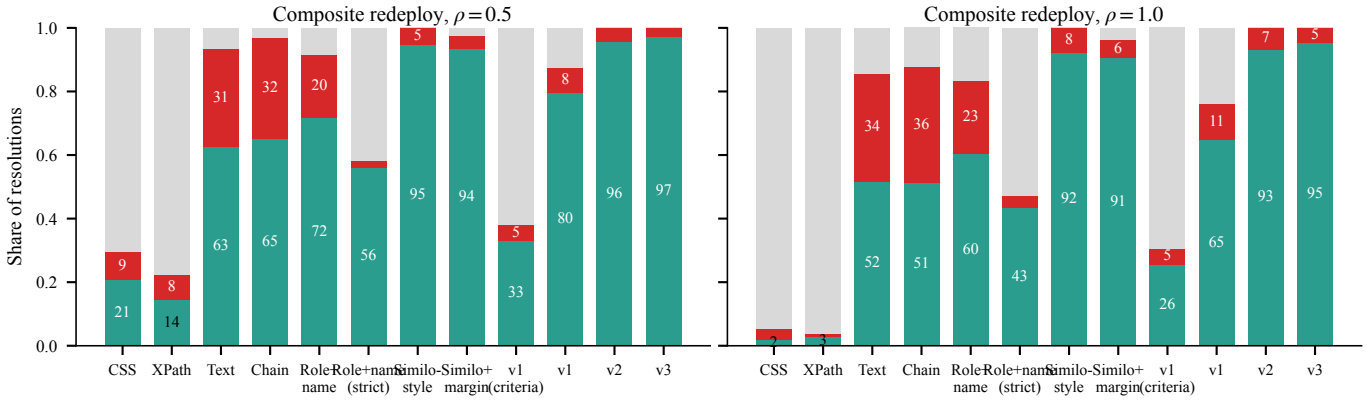


Fig. 4. Where each method's resolutions go under composite redeploy: correct, *wrong* (red), or abstain. Percentages are shown for the correct and wrong segments.

TABLE VI
CORRECT/WRONG PERCENT AT ρ=1 FOR EACH OPERATOR. BOLD MARKS THE HIGHEST CORRECT RATE IN EACH ROW.

Operator	CSS	XPath	Text	Chain	Role+name	Similo	Ours v1	Ours v2	Ours v3
Identity (ρ=0)	**100/0**	**100/0**	74/26	**100/0**	85/14	**100/0**	99/0	**100/0**	**100/0**
Class rename	25/0	**100/0**	74/26	83/17	84/15	**100/0**	97/0	**100/0**	**100/0**
ID randomisation	82/0	**100/0**	74/26	94/6	84/15	100/0	97/0	**100/0**	**100/0**
DOM reorder	61/29	31/25	74/26	68/32	83/14	95/5	88/7	94/6	**96/4**
Wrapper drift	25/11	0/0	73/27	74/26	84/15	99/1	98/0	99/1	**100/0**
Copy drift	**100/0**	**100/0**	33/20	**100/0**	35/13	98/2	39/4	98/2	100/0
Dup. (other section)	89/10	94/6	40/59	89/11	58/40	97/3	70/27	95/5	**99/1**
Dup. (adjacent)	89/11	89/11	74/26	89/11	84/14	85/15	94/4	98/2	**100/0**
Composite redeploy	2/3	3/1	52/34	51/36	60/23	92/8	65/11	93/7	**95/5**

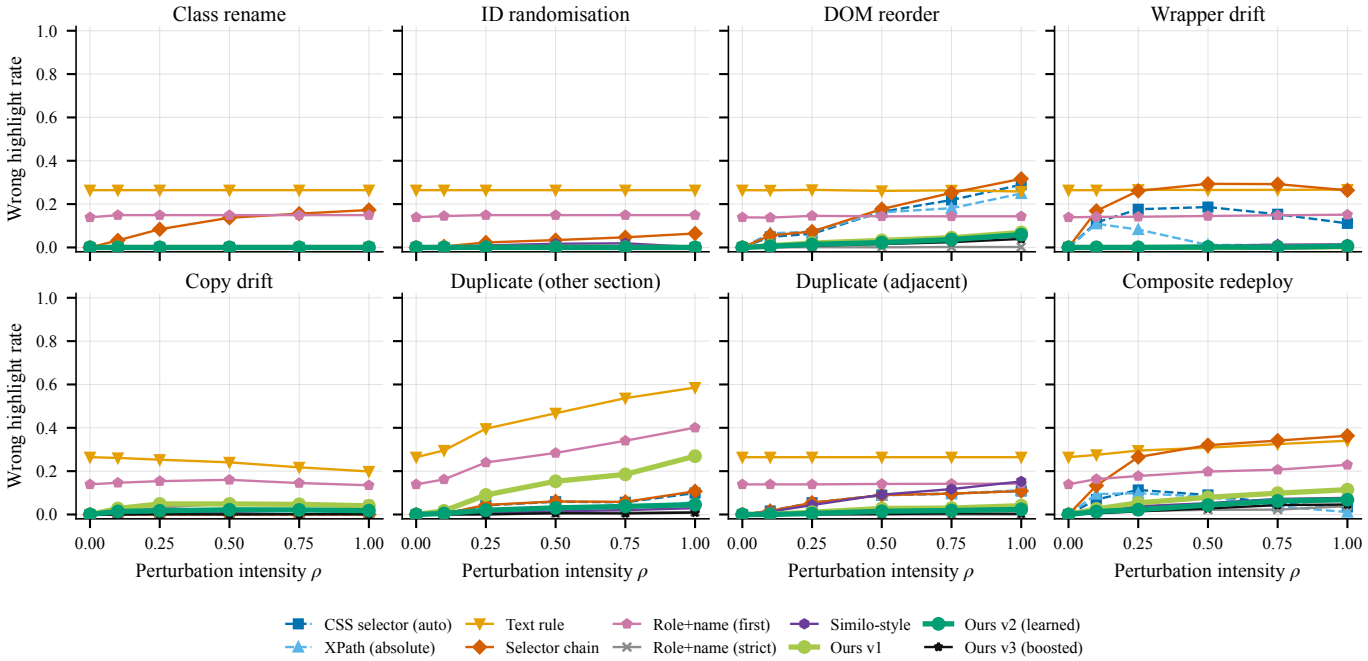


Fig. 5. Wrong-highlight rate against intensity. Always-answer baselines (text rule, chain, role+name) accumulate wrong highlights as the page drifts; abstaining methods stay near zero by giving up coverage.

MLP, and their ensemble reach 91.4%, 92.0%, and 92.2%, no better than the linear model. Combining the ranker with

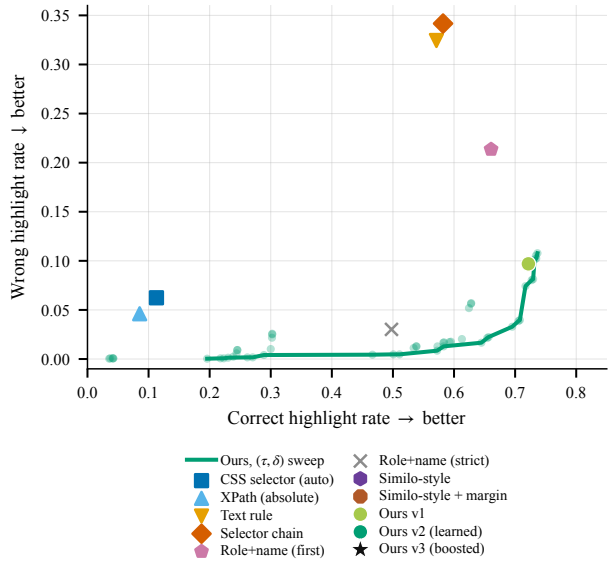


Fig. 6. Correct against wrong-highlight rate (composite redeploy, $\rho \in \{0.5, 1\}$). Lower right is better. The green curve sweeps (τ, δ) for our resolver; markers are baselines. The star is the held-out margin gate.

TABLE VII
DESCRIPTOR ABLATION, COMPOSITE REDEPLOY POOLED (PERCENT).

Variant	C	W	A
Full descriptor (shipped)	72.2	9.7	18.1
− section & ordinal hints	75.3	8.4	16.2
− bounding-box hint	70.3	9.7	20.1
− internal id	73.4	10.5	16.1
− all context (role, name, text, test id)	65.6	18.6	15.8
role + name only	65.1	19.2	15.7

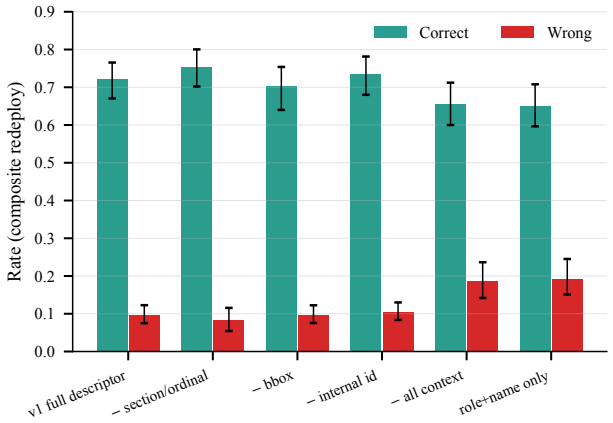


Fig. 7. Ablation with 95% cluster-bootstrap intervals. Removing all context nearly doubles wrong highlights.

temperature scaling, dual-softmax agreement between targets and candidates, Sinkhorn optimal transport, stable (Gale–Shapley) matching, per-site self-training, and a linear-plus-boosted ensemble all plateau at or below 94.0%, the value joint assignment alone gives. Adversarial augmentation that masks the text signals during training does not help (93.1–94.0%), and neither do fly-olfaction-inspired features, in which each element's attributes are hashed by sparse random expansion and winner-take-all [57] (93.6%); a seven-feature model built on them reaches 92.2%, close to Similo but not above it. Two further modern techniques did not help either: VON-style evidence from visually overlapping elements [18] (93.2%) and pretrained sentence embeddings of text, labels, and neighbours (93.1%). We read this as evidence that the remaining errors are limited by the information in the attributes (for example targets whose text and position both changed), not by the optimiser or the architecture.

Per-site view and the matcher's skeleton. Fig. 8 shows, for each of the 37 real websites, the correct rate of every published baseline (from the authors' tables), of Similo, and of our one-by-one and joint rankers. Our joint ranker is better than Similo on 15 sites, tied on 17, and worse on 5 (Intuit −36 points, Zillow −22, Amazon −10, Indeed −8, Twitter −5); the baselines that rely on a single locator (XPath, Id-XPath) score zero on most sites. Fig. 9 is the learned matcher's "skeleton": the signals with the largest total weight are word-level text overlap, text similarity, compression distance of the text, the id-based path similarity, the tag, position, and XPath similarity, which matches the failure analysis (text drift dominates the errors). Fig. 10 paints every element of three real pages by the probability our matcher assigns to it: probability mass concentrates on the true target (green) even where Similo picks a neighbour (red dashed), though it can be low (Amazon, P=0.16), where the margin gate abstains.

Fresh sites, zero-shot. To test beyond the 40 sites we built a new before/after set from Internet Archive snapshots of sites not in Similo's data (Python, W3C, Apache, Debian, Ubuntu, Mozilla, GNU), labelling ground truth from anchors that are then removed from every method's input (link target, or id), and keeping only targets whose XPath changed. We first checked our exact port of Similo's scoring (weights and similarity functions from the authors' Java source) on the original dataset: it scores 89.1% against the 90.1% the authors report, so we use it as the Similo baseline on the new data. The Internet Archive throttled our crawler, so the new set is small: with 2021 versus 2025 snapshots (4 sites, 92 targets) both methods are saturated (Similo 98.9%, ours 100%), and with 2017 versus 2025 snapshots (3 sites, 67 targets) ours scores 92.5% against 91.0% (+1.7 points, CI 0.0 to +5.0), trained only on the original sites. This confirms the direction on unseen sites but is too small to be evidence by itself.

F. Authored criteria versus descriptors

The shipped plan executor (criteria path) is much more conservative than the matcher: only 29.3% correct and 65.7% abstain under composite redeploys, with wide page-to-page spread. Exact-name equality and the hard section constraint of Eq. (2) are the cause; on real pages the section label can be the landmark's entire text content, which changes whenever anything inside it does. This is the clearest engineering con-

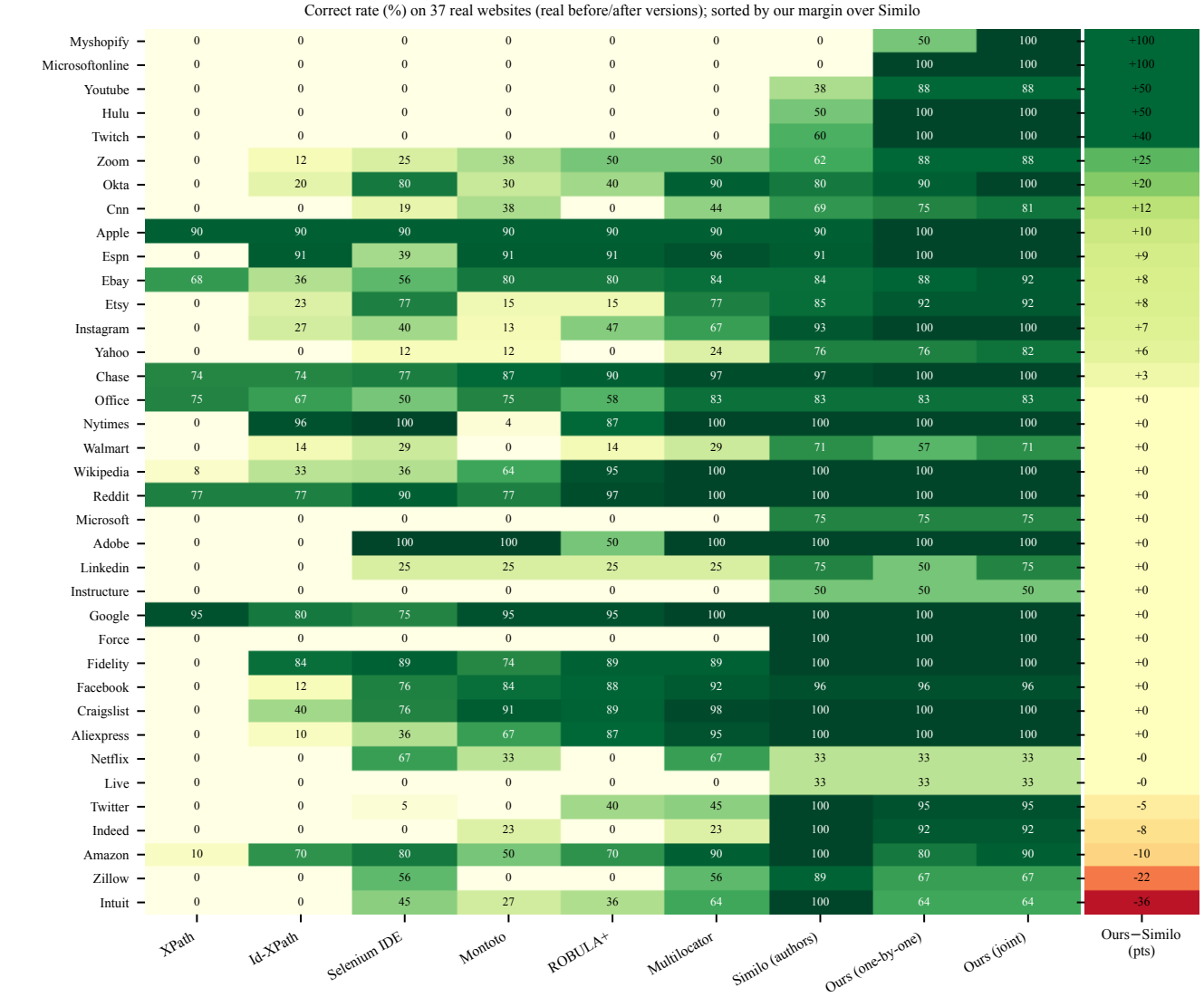


Fig. 8. Per-website correct rate on the 37 real before/after sites: published baselines, Similo, and ours (held-out sites). Rows are sorted by our margin over Similo (right column); green rows at the top are where we outperform, red rows at the bottom where we lose.

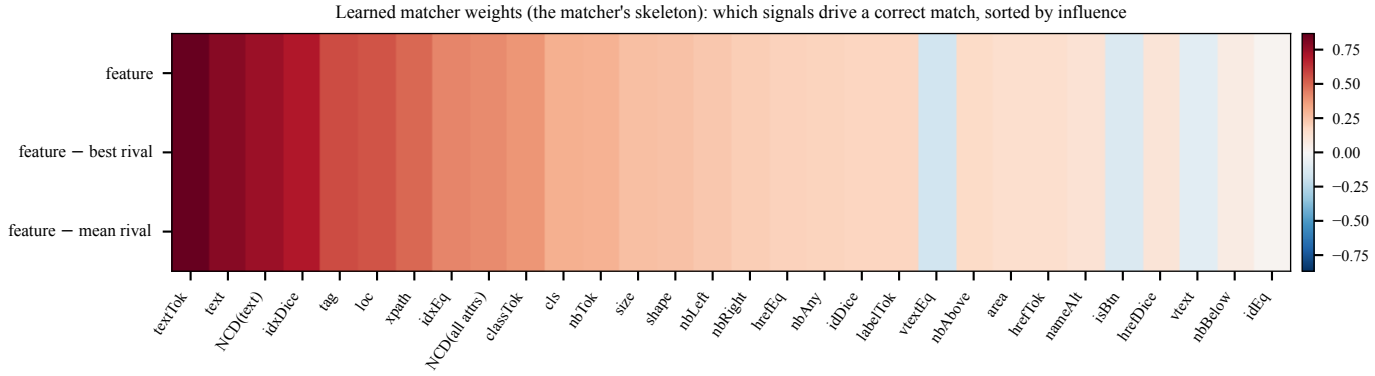


Fig. 9. Learned matcher weights over the 30 locator and semantic features, with context variants (feature minus the best and mean rival candidate). Text-drift-robust signals dominate.

clusion of the study: the criteria path should adopt descriptor scoring and the margin gate.



Fig. 10. Layout heatmaps of three real pages (zoomed on the target): every element is coloured by the probability our matcher assigns to it. The true target is outlined green; Similo's wrong pick is red dashed.

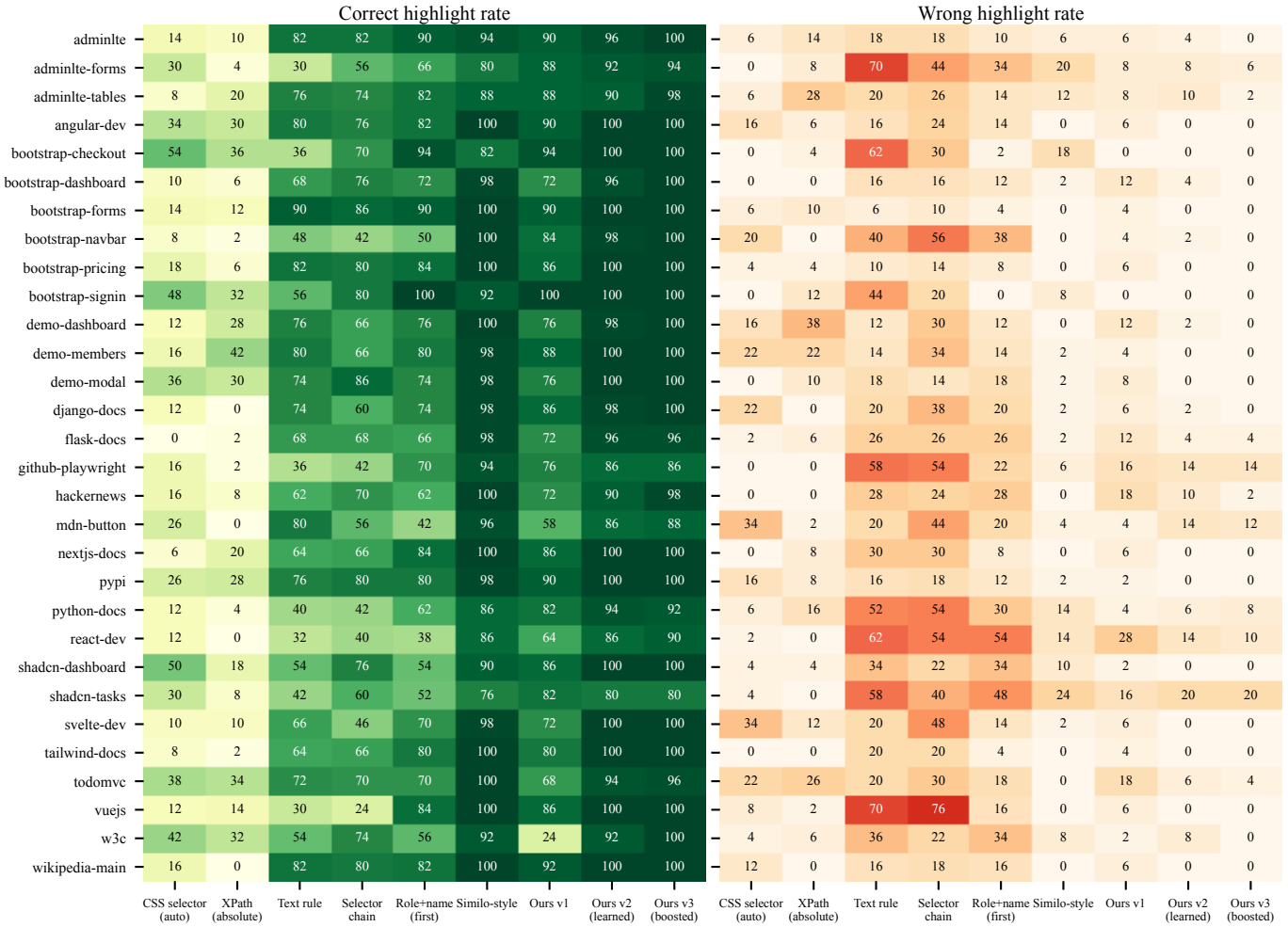


Fig. 11. Per-page correct (left) and wrong (right) rates at composite redeploy ρ=0.5. Improvements are broad, not driven by one page; our fixture is not special. Wrong rates remain non-trivial on documentation pages with many near-identical links.

G. Runtime cost

Fig. 12(a) shows that a tick (extraction plus one score pass) takes a median of 7.8 ms across the 14 timed snapshots and

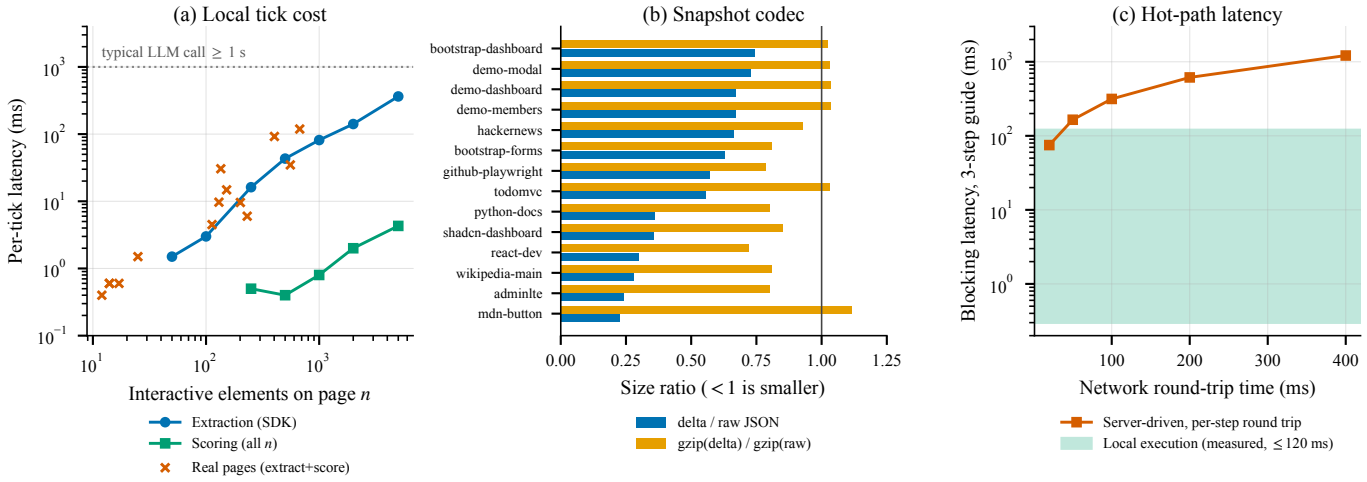


Fig. 12. (a) Measured per-tick latency: extraction (DOM and style reads) dominates and grows with page size, scoring stays small; real pages are crosses. (b) Snapshot-codec size relative to raw JSON, and after gzip. (c) Blocking latency of a 3-step guide: server-driven is a *model* (steps × (RTT + 5 ms measured server time)); the green band is the measured worst local tick.

TABLE VIII
REAL BEFORE/AFTER DATA (SIMILO REPLICATION PACKAGE, 40 SITES, 598 TARGETS; PERCENT). BASELINE ROWS ARE THE AUTHORS' PUBLISHED OUTPUTS; W IS WRONG, A IS NOT FOUND/ABSTAIN. †PROPOSED.

Method	C		W	A
XPath	20.9	[9.1, 34.5]	3.0	76.1
Id-XPath	40.8	[26.9, 53.6]	7.4	51.8
Selenium IDE	53.3	[41.9, 63.4]	6.2	40.5
Montoto	54.3	[40.0, 65.3]	1.3	44.3
ROBULA+ [16]	64.5	[50.0, 75.2]	4.0	31.4
Multilocator	76.1	[64.0, 84.7]	0.0	23.9
Similo (authors' code) [17]	90.1	[84.4, 94.2]	9.9	0.0
Ours: boosted ranker (v3)	88.9	[83.5, 93.0]	11.1	-0.0
Ours: v3 + margin gate†	82.8	[75.2, 88.4]	5.4	11.7
Ours: listwise linear ranker	91.7	[86.8, 95.0]	8.3	0.0
Ours: linear + online feedback	91.9	[87.1, 95.1]	8.1	0.0

at most 119 ms (the Wikipedia main page: 673 interactive elements, 250 kept). On synthetic pages extraction grows to 82 ms at 1,000 and 364 ms at 5,000 interactive elements, while scoring all 5,000 takes 4.3 ms; the cap of 250 therefore bounds scoring but not extraction. Even the worst measured tick is far below the typical latency of a model call.

The snapshot codec (Fig. 12(b)) is lossless on all 14 timed snapshots (decode equals input) and shrinks raw JSON to a median of 56% (range 23–74%). After gzip, however, the median is 89% and the worst case is 111%, so the codec adds little once transport compression is on. We report this plainly; the codec's value is mostly in memory and parse cost, which we did not measure.

We walked the real invite flow through the real server (3 steps). The guidance path made one session-start request (5,174 B) and three fire-and-forget checkpoints (532, 246 and 242 B), with no blocking call on any step transition. A design that asks a server for every step needs at least one blocking round trip per step; Fig. 12(c) models that cost. This is a model, not a measurement of a competing product.

H. Engineering verification

On the evaluated revision, all 23 shared and 36 server unit tests pass. The Playwright end-to-end suite (ten specification files covering duplicate buttons, forced remounts, portal modals, SPA routes, refresh-resume, placement, and the composer) was not re-run for this study, since it needs a bundled Chromium; the benchmark above used the system Chrome instead. Our fixture was written by us and may favour our design, which is why half of the corpus is public pages.

VI. DISCUSSION AND LIMITATIONS

What the results do and do not show. They show that, under controlled structural drift, multi-signal matching beats the locator strategies that DAPs document, that our hand-weighted v1 was not competitive with a Similo-style matcher, that a linear learned ranker (v2) ties it, and that a boosted ranker (v3) is significantly better than it on held-out pages. They do not show that we beat the published state of the art, since our Similo-style arm is a simplified reimplementation and not the authors' code; v3 was not run in the leave-one-operator-out protocol, and and they do not show that our product beats Whatfix, WalkMe or any vendor: the baselines are our reimplementations of documented behaviour, vendors may ship undocumented repair, and the commercial platforms offer analytics, localisation and integrations that we do not.

Scale and realism. The corpus is 14 page states and the changes are synthetic; the strongest next validation is real before/after release pairs of the same site, and a run of the original Similo implementation. **Perturbations are synthetic.** Real redeploys differ from our operators, and the seeded composite weights are our choice. We therefore plot every intensity and operator separately (Figs. 3, 5). The Similo-style

arm is our simplified reimplementation, not the authors' code; a run of the original is needed before any claim against Similo itself. v2 is trained and tested on synthetic perturbations of the same operator family (we mitigate with page-held-out and operator-held-out splits). Pages are static snapshots without scripts, so dynamic behaviour (late rendering, shadow DOM, cross-origin frames) is out of scope.

Known weaknesses. Exact name matching makes copy drift costly. The ordinal hint is positional. Section labels on real pages can be very long landmark texts. Extraction is capped at 250 elements, and its cost scales with page size. The language-model compiler is not quantitatively evaluated here: we have no live-API experiment, and the validation guard is covered only by unit tests.

A defect found by the evaluation. While counting requests we observed the SDK calling `GET /v1/onboarding` about eight times per second *indefinitely* after a flow completes: the completion handler refreshes the discovery dock on every tick, and re-rendering the dock mutates the page and triggers the next tick. It does not affect the guidance-path numbers above (we excluded those requests), but it is a real inefficiency to fix.

Statistical scope. With 14 page states and a cluster bootstrap, intervals are honest but wide for page-sensitive arms (the criteria path especially). The margin-gate δ is cross-fitted, but both folds chose the same value from a coarse grid.

VII. Conclusion

Guidance should not remember where a button was; it should work out which button is meant, and say so when it cannot. On 30 page states and 60,475 resolutions, our first hand-weighted resolver beat documented DAP locator strategies but not a Similo-style weighted-similarity matcher. A linear learned ranker (v2) tied that matcher, and a boosted ranker (v3) on candidate-context features, tested on held-out pages, beat it significantly (96.3% against 93.6% correct; 3.7% against 6.4% wrong). On the real Similo dataset the boosted ranker ties the authors' original code and the margin gate trades coverage for fewer wrong highlights; online feedback gave a negligible gain. Caveats remain: the synthetic-benchmark baseline is our reimplementation, the perturbations are synthetic, and v3 is not yet in the SDK. Longer feedback streams, a leave-one-operator-out run for v3, and in-browser latency measurements are the next experiments. Code, harness, and raw results are in the project repository [58].

References

[1] A. Medvedev, "finder: CSS selector generator," GitHub repository, npm package `@medv/finder` v3.2.0, accessed: Oct. 15, 2026. [Online]. Available: https://github.com/antonmedv/finder

[2] WalkMe, "What is WalkMe?" WalkMe Help Center, accessed: Oct. 7, 2026. [Online]. Available: https://support.walkme.com/knowledge-base/what-is-walkme/

[3] Pendo, "Use CSS rules for guide targeting," Pendo Support, accessed: Oct. 7, 2026. [Online]. Available: https://support.pendo.io/hc/en-us/articles/28618614571547

[4] Appcues, "Troubleshoot element targeting," Appcues Docs, accessed: Oct. 7, 2026. [Online]. Available: https://docs.appcues.com/flows/css-selectors

[5] Userpilot, "Select the right element," Userpilot Docs, accessed: Oct. 7, 2026. [Online]. Available: https://docs.userpilot.com/ce-basics/select-element

[6] Userflow, "Tooltip element selection," Userflow Help Center, accessed: Oct. 7, 2026. [Online]. Available: https://help.userflow.com/docs/tooltip-element-selection.md

[7] M. Hammoudi, G. Rothermel, and P. Tonella, "Why do record/replay tests of web applications break?" in *Proc. IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, Chicago, IL, USA, 2016, pp. 180–190.

[8] WalkMe, "WalkMe can't find an element on the page," WalkMe Help Center, accessed: Oct. 7, 2026. [Online]. Available: https://support.walkme.com/knowledge-base/walkme-cant-find-an-element-on-the-page/

[9] ——, "Advanced settings," WalkMe Help Center, accessed: Oct. 7, 2026. [Online]. Available: https://support.walkme.com/knowledge-base/behavior/

[10] Whatfix, "Improve element selection using visual cues," Whatfix Support, accessed: Oct. 7, 2026. [Online]. Available: https://support.whatfix.com/docs/improve-element-selection-using-visual-cues

[11] Pendo Community, "Guide target element matches multiple," Pendo Support Community, accessed: Oct. 7, 2026. [Online]. Available: https://support.pendo.io/hc/en-us/community/posts/5902685378971-Guide-target-element-matches-multiple

[12] Appcues, "Check if Appcues uses a selector before changing your code," Appcues Docs, accessed: Oct. 7, 2026. [Online]. Available: https://docs.appcues.com/installation-overview/check-selector-dependencies-before-code-changes

[13] Userpilot, "Userpilot error reference," Userpilot Docs, accessed: Oct. 7, 2026. [Online]. Available: https://docs.userpilot.com/reference/errors.md

[14] Whatfix, "Introducing Whatfix AI," Whatfix Blog, accessed: Oct. 7, 2026. [Online]. Available: https://whatfix.com/blog/introducing-whatfix-ai/

[15] IT Brief New Zealand, "Whatfix unveils AI agents to boost enterprise user productivity," accessed: Oct. 7, 2026. [Online]. Available: https://itbrief.co.nz/story/whatfix-unveils-ai-agents-to-boost-enterprise-user-productivity

[16] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "ROBULA+: An algorithm for generating robust XPath locators for web testing," *Journal of Software: Evolution and Process*, vol. 28, no. 3, pp. 177–204, 2016. [Online]. Available: https://doi.org/10.1002/smr.1771

[17] M. Nass, E. Alégroth, R. Feldt, M. Leotta, and F. Ricca, "Similarity-based web element localization for robust test automation," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 3, 2023. [Online]. Available: https://doi.org/10.1145/3571855

[18] M. Nass, E. Alégroth, R. Feldt, and R. Coppola, "Robust web element identification for evolving applications by considering visual overlaps," arXiv preprint arXiv:2301.03863 (VON Similo), 2023. [Online]. Available: https://arxiv.org/abs/2301.03863

[19] M. Nass, E. Alégroth, and R. Feldt, "Improving web element localization by using a large language model," arXiv preprint arXiv:2310.02046, 2023. [Online]. Available: https://arxiv.org/abs/2310.02046

[20] A. Kluge and A. Stocco, "Web element relocalization in evolving web applications: A comparative analysis and extension study," arXiv preprint arXiv:2505.16424 (HybridSimilo), 2025. [Online]. Available: https://arxiv.org/abs/2505.16424

[21] ReproBreak authors, "ReproBreak: A dataset of reproducible web locator breaks," arXiv preprint arXiv:2605.12158, 2026. [Online]. Available: https://arxiv.org/abs/2605.12158

[22] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a generalist agent for the web," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. [Online]. Available: https://arxiv.org/abs/2306.06070

[23] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist web agent, if grounded," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024. [Online]. Available: https://arxiv.org/abs/2401.01614

[24] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[25] C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970.

[26] W3C, "Accessible name and description computation 1.2," W3C Recommendation, accessed: Oct. 15, 2026. [Online]. Available: https://www.w3.org/TR/accname-1.2/

[27] G. Comanici *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025. [Online]. Available: https://arxiv.org/abs/2507.06261

[28] AdminLTE, "AdminLTE 3 dashboard v3 demo," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://adminlte.io/themes/v3/index3.html

[29] ——, "Adminlte general forms," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://adminlte.io/themes/v3/pages/forms/general.html

[30] ——, "Adminlte data tables," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://adminlte.io/themes/v3/pages/tables/data.html

[31] Angular, "Angular home," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://angular.dev/

[32] Bootstrap, "Bootstrap checkout example," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/examples/checkout/

[33] ——, "Bootstrap dashboard example," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/examples/dashboard/

[34] ——, "Bootstrap forms overview," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/forms/overview/

[35] ——, "Bootstrap navbar component docs," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/components/navbar/

[36] ——, "Bootstrap pricing example," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/examples/pricing/

[37] ——, "Bootstrap sign-in example," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://getbootstrap.com/docs/5.3/examples/sign-in/

[38] Django, "Django documentation," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://docs.djangoproject.com/en/stable/

[39] Flask, "Flask documentation," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://flask.palletsprojects.com/en/stable/

[40] Microsoft, "microsoft/playwright repository page," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://github.com/microsoft/playwright

[41] Y Combinator, "Hacker news," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://news.ycombinator.com/

[42] MDN Web Docs, "`<button>`: The button element," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/button

[43] Next.js, "Next.js documentation," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://nextjs.org/docs

[44] PyPI, "Pypi home," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://pypi.org/

[45] Python Software Foundation, "Python 3 documentation," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://docs.python.org/3/

[46] Meta Open Source, "React documentation," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://react.dev/

[47] shadcn/ui, "Dashboard example," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://ui.shadcn.com/examples/dashboard

[48] ——, "shadcn/ui tasks example," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://ui.shadcn.com/examples/tasks

[49] Svelte, "Svelte home," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://svelte.dev/

[50] Tailwind, "Tailwind css installation docs," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://tailwindcss.com/docs/installation

[51] TodoMVC, "TodoMVC (react example)," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://todomvc.com/examples/react/dist/

[52] Vue.js, "Vue.js home," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://vuejs.org/

[53] W3C, "W3c home," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://www.w3.org/

[54] Wikipedia, "Main page," Web page, accessed: Oct. 15, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Main_Page

[55] Microsoft, "Playwright: Locators (`getbyrole`)," Playwright documentation, accessed: Oct. 15, 2026. [Online]. Available: https://playwright.dev/docs/locators

[56] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[57] S. Dasgupta, C. F. Stevens, and S. Navlakha, "A neural algorithm for a fundamental computing problem," *Science*, vol. 358, no. 6364, pp. 793–799, 2017. [Online]. Available: https://doi.org/10.1126/science.aam9868

[58] S. Satyam, K. Yadav, A. Razy, S. S. Bal, and V. T, "Onboarding copilot: source code, benchmark harness and results," GitHub repository, `paper/` directory, 2026. [Online]. Available: https://github.com/RazyAnas/onboarding-copilot-submission